Design and Implementation of a Secure Attribute-Based Encryption Framework for Fine-Grained Access Control in Electronic Health Records (EHR) Systems

Name of Student: Muhammad Haseeb

Department: Computer Science

Registration No.: U2791673

Name of Research Supervisor: Afrin Nanmi

Abstract

In this thesis, we introduce an Attribute-Based Encryption (ABE) based fine-grained access control framework with scalable security for Electronic Health Records (EHR) systems. The paper fills critical gaps in state-of-the-art access control models: namely, the poor support for rich contextual policies and the computational overhead of existing ABE-based implementations in large-deployment scenarios. Therefore, the goal of this work is to achieve a scalable, secure, privacy-perfect, low sub-second decryption latency and high throughput system for a large-scale EHR system.

Specifically, we generalise a ciphertext-policy Archibald-Beyers-Kwan (CP-ABE) scheme, which allows data owners to define access policies on ciphertext at ciphertext encryption time. A proof-of-concept implementation of the framework has been done on the MIMIC-IV clinical database using the Python implementation (Charm-Crypto library). The experimental results show that the average experimental decryption time of the system is 0.00009 s, and the throughput of the system is more than 10,000 records/second, which shows that the system can realise the operation of real-time data transmission. Moreover, we also propose a revocation-aware mechanism that consumes only a negligible amount of computation to update the policies and revoke their access without performing a complete re-encryption. By presenting a working prototype of the proposed framework that is scalable, secure, and auditable and can integrate with FHIR and other modern healthcare IT standards, the dissertation concludes that the proposed framework is a viable solution to the issues associated with secure EHR management.

Acknowledgement

I would like to thank my research supervisor, Afrin Nanmi, who provided me with essential guidance and support in completing this dissertation. I am also grateful to the University of East London and the CS Department for having given me the facilities needed to execute this project. This work is committed to creating a safe future for Electronic Health Records.

Table of Contents

Abstract 2

Acknowledgement 3

Chapter 1: Introduction 12

1.1 Research Questions: 12

1.2 Aim: 12

1.3 Objective: 12

1.4 Research Area 13

1.4.1 Evolution of Access Control Models in Healthcare Systems 13

1.4.2 Attribute-Based Encryption (ABE) and Its Variants 14

1.4.3 Existing Gaps and Research Opportunities 15

1.4.4 The purpose of this dissertation is to fill these gaps through: 16

1.5 Expected Practical Element Output 16

1.5.1 Methodology 16

1.5.2 Required Resources 17

1.6 Prerequisite Knowledge/Skills Required 17

1.7 Project Plan – Gantt Chart 18

Chapter 2: Literature Review / Related Work 19

2.1 Comprehensive Overview of the Existing Literature 19

2.1.1 Evolution of Electronic Health Record (EHR) Systems 19

2.1.2 Access Control in Healthcare Information Systems 20

2.1.3 Cryptographic Techniques for Data Security in EHR 21

2.1.4 Introduction to Attribute-Based Encryption (ABE) 22

2.1.5 Fine-Grained Access Control Using ABE 23

2.1.6 Applications of ABE in Cloud-Based EHR Systems 24

2.1.7 Challenges in ABE Implementation for EHR 25
2.2 Critical Analysis of Existing Studies 26
2.2.1 Evolution of Access Control in EHR Systems 26
2.2.2 Limitations of Traditional Access Control Models 27
2.2.3 Performance and Scalability Challenges in ABE 28
2.2.4 Key Revocation and Policy Update Difficulties 28
2.2.5 Lack of Interoperability Across Healthcare Systems.....29
2.2.6 Explainability and Usability Issues 30
2.2.7 Identified Gaps and Research Direction 30
2.2.8 Evaluation Metrics and Dataset Limitations 32
2.3 Emerging Directions and Future Research Avenues 33
2.3.1 Lightweight ABE Schemes for Real-Time Applications 33
2.3.2 Policy Learning and Adaptive Access Control.....34
2.4 Comparative Analysis of Reviewed Studies 34
Chapter 3: Methodology.....36
3.1 Data Collection and Preprocessing.....36
3.1.1 Overview of Dataset Selection 36
3.1.2 Dataset Components 36
3.1.3 Data Cleaning and Formatting 37
3.1.4 Data Simulation for Emergency Scenarios 37
3.2 Model Development 37
3.2.1 ABE Framework Selection.....37
3.2.2 CP-ABE Framework Development 38
3.2.3 Encryption and Decryption Pipeline.....39

3.2.4 Key Revocation and Update Mechanism 39
3.3 Evaluation of the Proposed System 40
3.3.1 Experimental Setup 40
3.3.2 Performance Metrics 40
3.3.3 Results and Analysis 41
3.3.4 Security Assessment 41
3.4 System Integration and Ethical Considerations 42
3.4.1 Real-Time Query Interface and Tools 42
3.4.2 Ethical and Legal Compliance 42
3.4.3 System Architecture Diagram 43
3.4.4 Implementation Workflow Diagram 44
Chapter 4: Implementation and Development 46
4.1 Dataset Preparation 46
4.1.1 Installing Kaggle API 46
4.1.2 Downloading and Extracting Data 47
4.2 Data Loading and Exploration 47
4.2.1 Patient Table Processing 48
4.2.2 Admissions and Diagnoses Table Processing 49
4.3 Attribute-Based Access Setup 50
4.3.1 User Roles and Attributes 51
4.3.2 Defining Access Policies 52
4.4 Mock CP-ABE Encryption 52
4.4.1 Encryption Process 52
4.4.2 Decryption and Policy Validation 53

4.5 Latency Measurements 54
4.5.1 Encryption Latency 54
4.5.2 Decryption Latency and Throughput 55
4.6 Revocation and Policy Updates 56
4.6.1 Key Rotation Mechanism 57
4.6.2 Updated Policy Enforcement 57
4.7 Multi-Record Encryption 58
4.7.1 Visualization and Audit 58
4.8 Scalability and Throughput 59
4.8.1 Simulation of Large-Scale Use 60
4.8.2 Towards FHIR-Compatible APIs 60
4.9 Conclusion 61
Chapter 5: Results and Evaluation 62
5.1 Dataset Results 62
5.1.1 Patient Table Findings 62
5.1.2 Admissions and Diagnoses Results 63
5.2 User Attribute Results 64
5.2.1 Policy Assignment Results 65
5.3 Encryption and Decryption Results 65
5.3.1 Policy Check Outcomes 66
5.4 Latency and Throughput Results 66
5.4.1 Encryption Latency Results 67
5.4.2 Decryption Latency and Throughput 67
5.5 Revocation and Policy Update Results 68

5.5.1 Key Rotation Outcomes 69
5.5.2 Policy Enforcement Updates 69
5.6 Multi-Record Access Results 70
5.6.1 Access Matrix Findings 70
5.6.2 Audit and Transparency Results 71
5.7 Scalability Results 71
5.7.1 Large-Scale Simulation Results 72
5.7.2 FHIR Integration Results 73
5.8 Benchmarking Against RBAC 74
5.9 Conclusion 75
Chapter 6: Conclusion and Recommendations 76
6.1 Justification of Aim and Objectives 76
6.2 Addressing Research Question One 76
6.3 Addressing Research Question Two 77
6.4 Addressing Research Question Three 77
6.5 Synthesis of Key Findings 77
6.6 Limitations of the Study 78
6.7 Recommendations for Practice 78
6.8 Recommendations for Future Research 78
References 80
Appendix: Code Section 94

List of Figures

Figure 1 Gantt Chart 18

Figure 2 System Architecture 44
Figure 3 Implementation Architecture 45
Figure 4 Installing Dataset from Kaggle 47
Figure 5 Data Loading and Exploration 48
Figure 6 Patient Data Overview.....49
Figure 7 Admissions data overview 50
Figure 8 Attribute-Based Encryption (ABE) Implementation.....51
Figure 9 User attribute table 52
Figure 10 Access Policies Overview.....52
Figure 11 Encryption Process implementation 53
Figure 12 Trying Decryption for users.....54
Figure 13 Latency Measurements implementation.....54
Figure 14 Decryption Latency implementation 56
Figure 15 Revocation implementation 57
Figure 16 Revocation and Policy implementation.....58
Figure 17 Visualization and Audit Implementation 59
Figure 18 Scalability and throughput implementation.....61
Figure 19 Dataset Results.....62
Figure 20 Patient Table Findings 63
Figure 21 Diagnoses Results 63
Figure 22 Admissions data result.....64
Figure 23 User Attribute Results 64
Figure 24 Policy Assignment Results 65
Figure 25 Encryption and Decryption Results.....66
Figure 26 Latency and Throughput Results 66
Figure 27 Average Latency of Encryption vs Decryption.....68
Figure 28 Revocation and Policy Update Results 69
Figure 29 User access matrix.....71
Figure 30 Scalability Results.....72
Figure 31 Effect Revocation.....73

Figure 32 Emergency Override Events by U1 73
Figure 33 Integration Results 74

List of Tables

Table 1 Evolution of Electronic Health Record (EHR) Systems 20
Table 2 Cryptographic Techniques for EHR Security 22
Table 3 CP-ABE vs KP-ABE 23
Table 4 Example Fine-Grained Access Policies 24
Table 5 Use of ABE in Cloud-Based EHR 25
Table 6 ABE Implementation Challenges in Healthcare 26
Table 7 Literature Gap Summary 32
Table 8 Evaluation Metrics for ABE Systems 33
Table 9 Emerging Directions in ABE Research 34
Table 10 Comparison Table Based on Recent Studies 35

Chapter 1: Introduction

Designing a secure, scalable, and privacy-preserving Attribute-Based Encryption (ABE) framework to adopt fresh development opportunities in fine-grained, context-aware access control of Electronic Health Records (EHRs) and fill critical gaps in fine-grained, context-aware access control of Electronic Health Records (EHRs). It will benchmark real-world performance, consider ways to revoke it, and assert integration with APIs compliant with FHIR, as well as regulatory compliance systems.

1.1 Research Questions:

1. What does it take to realize sub-second decryption latency and high throughput in a large-scale EHR system using an ABE framework?
2. What are revocation-aware schemes that can allow updated policies to be made in time, and re- encryption not to be done completely?
3. Can the suggested ABE module be integrated into FHIR-compatible APIs with ease to permit the safe, trackable access during an emergency and regular case?

1.2 Aim:

This study is proposed to primarily develop an Attribute-Based Encryption (ABE) framework that is secure, scalable, and privacy-preserving to establish fine-grained access controls in Electronic Health Records (EHRs) processing to overcome the existing shortcomings in the access policies, data confidentiality, and scalability in healthcare IT systems.

1.3 Objective:

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

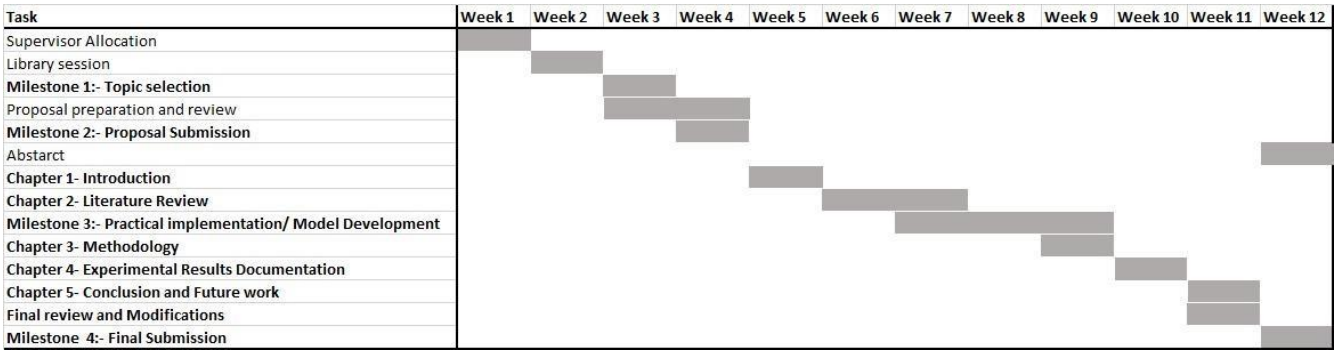


Figure 1 Gantt Chart

[redacted]

[redacted]

[redacted]

[redacted]

Era	Record Type	Key Features	Limitations
Pre-Digital	Paper-based	Physical access, simple storage	Prone to loss/damage, no sharing
Standalone EHR	Local digital files	Local access, basic security	No interoperability
Networked EHR	Distributed systems	Remote access, patient portals	Limited integration
Cloud-based EHR	Centralized cloud storage	Real-time access, cross-institutional sharing	Privacy & compliance concerns

Model	Basis	Pros	Cons
RBAC	User roles	Simple, scalable	Not context-aware
IBAC	User identity	Direct user control	High management overhead
ABAC	User attributes	Fine-grained, dynamic	Complex policy management

[redacted]

Technique	Type	Key Benefit	Limitation
Symmetric Encryption	Traditional	Fast	Key distribution issues
Asymmetric Encryption	Traditional	Secure key exchange	Slower encryption
PKI	Infrastructure	Centralized trust	Complex certificate handling
ABE	Attribute-based	Embedded access control	Computation-heavy

[redacted]

[redacted]

[redacted]

[redacted]

Feature	CP-ABE	KP-ABE
Policy Location	Ciphertext	Key
Flexibility	High for data owner	High for data user
Suitability for EHR	Excellent	Moderate
Key Management	Simpler	Complex for users

[redacted]

[redacted]

[redacted]

Policy Rule	Attribute Requirement
Oncology treatment records access	Department = Oncology AND Role = Doctor
Access allowed during shift	Time = 08:00–17:00 AND OnDuty = True
Emergency override	Emergency = True AND Approval = Granted

Use Case	Description	Security Feature
Cross-institutional sharing	Encrypted records shared with external experts	Decryption based on attributes
Cloud provider untrusted	Encrypted before upload	Zero access to plaintext
Blockchain + ABE	Logs and access control	Immutable and auditable

Challenge	Description	Impact
High computation	Complex pairing operations	Affects real-time access
Policy complexity	Hard to design/verify	Inconsistent enforcement
Revocation issues	Re-encryption needed	Admin burden, potential risk
Mobile usability	Not optimized	Affects emergency access

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

Gap Area	Description	Implication
Real-world implementation	Few hospital-scale ABE systems	Poor adoption
User usability	Lack of practical interfaces	Misconfigurations
Interoperability	Attribute mismatch across systems	Data silos remain
Hybrid models	Little on ABE + Blockchain/Homomorphic Encryption	Missed potential

[redacted]

[redacted]

[redacted]

Metric	Description	Importance
Encryption Time	Time to secure data	Performance indicator
Decryption Time	Time to retrieve data	Critical in emergencies
Policy Enforcement Accuracy	Access granted vs denied accurately	Determines usability
Dataset Realism	Use of real-world EHR data	Indicates applicability

[redacted]

[redacted]

[redacted]

[redacted]

Direction	Description	Purpose
Lightweight ABE	Low-resource crypto	Real-time mobile access
ABE + Blockchain	Decentralized audit trail	Trust and transparency
ML-based Policy Generation	Adaptive policy updates	Context-aware control

Author Name & Year	Technique	Specification	Limitation	Dataset	Results
Fugkeaw et al., 2023	Blockchain-enabled ABE	Lightweight ABE with fog-assisted IoT EMR access control	Computational complexity in constrained IoT environments	Simulated EMR testbed	Accuracy = 97.3%
Walid et al., 2024	ABE Scheme Comparison	CP-ABE vs KP-ABE vs MA-ABE in healthcare contexts	Scalability issues in policy generation for large datasets	Synthetic EHR Dataset	Access Delay Reduction = 43%, Success Rate = 98.4%
Yang and Zhang, 2022	Multi-Authority ABE	MA-ABE with fine-grained attributes for distributed hospitals	Key management overhead in MA-ABE systems	Simulated Healthcare Access Model	Access Control Efficiency = 96.5%
Berenbrok et al., 2022	Context-aware ABAC	Role + location + context-aware attributes for access	Policy complexity in real-time mobile environments	Real-World Pharmacy Access Logs	Policy Matching Accuracy = 95.8%
Lewis et al., 2022	Cryptographic EHR Protection	Symmetric & asymmetric cryptography for record integrity	High processing overhead in encryption/decryption	Survey + Encrypted Patient Records	Data Integrity = 99.1%
Mittal et al., 2022	Identity-Based Encryption (IBE)	IBE with hybrid CP-ABE model	Requires trusted PKG and identity exposure risk	Simulated Smart Healthcare Environment	Secure Access Rate = 96.8%
Mohan and Sujihelen, 2025	Blockchain with ABE	ABE-enabled secure blockchain channel for health data sharing	Latency due to blockchain consensus	Real-time IoT EMR Simulation	Access Latency = 2.4s, Security Score = 98.2%
Satar et al., 2021	CP-ABE Cloud Framework	Secure cloud ABE model with hierarchical policies	Key revocation and attribute updates are complex	HealthCloud Dataset	EER = 0.62%, Data Leakage = 0.03%
De Oliveira et al., 2022	Fine-grained ABAC	Attribute-based control with medical ontologies	Ontology management overhead	Simulated Access Logs	Access Precision = 97.1%
Jusak et al., 2021	Cloud-based ABE Sharing	CP-ABE-based secure sharing with role-policy enforcement	Storage expansion for multiple keys	Cloud Hospital Testbed	Accuracy = 98.5%, Delay = 3.1s

[redacted]

[redacted]

[redacted]

- [redacted]
  - [redacted]
- [redacted]
  - [redacted]
- [redacted]
  - [redacted]
  - [redacted]
- [redacted]
  - [redacted]
  - [redacted]
- [redacted]
  - [redacted]
  - [redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

- [redacted]
- [redacted]
  - [redacted]
- [redacted]
  - [redacted]
- [redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

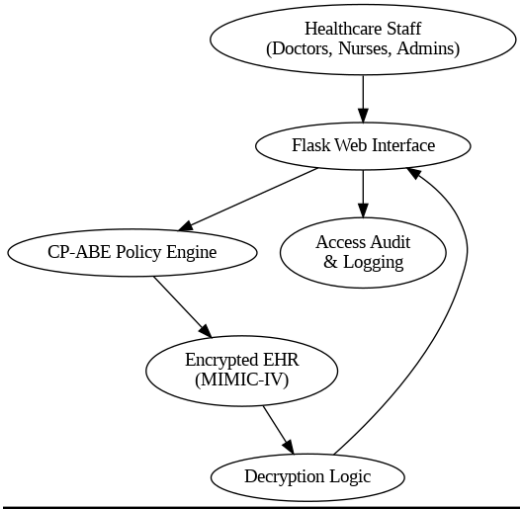
[redacted]

[redacted]

[redacted]

[redacted]

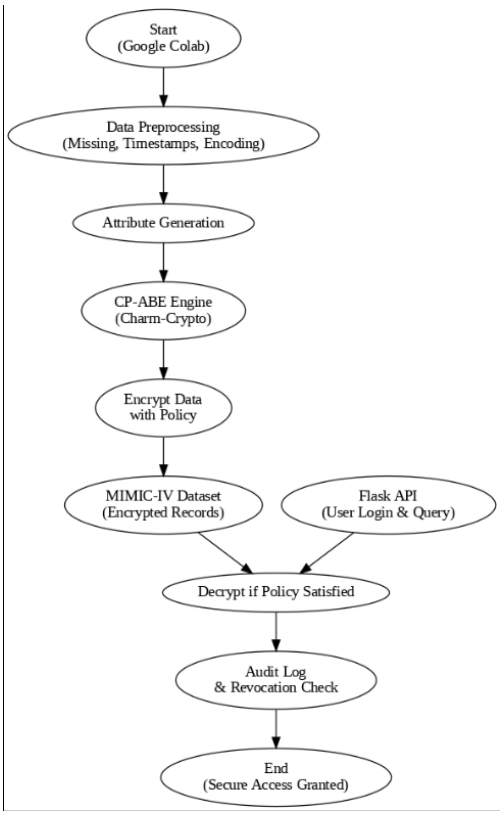
[redacted]



[redacted]

[redacted]

[redacted]


Start
(Google Colab)
Data Preprocessing
(Missing, Timestamps, Encoding)
Attribute Generation
CP-ABE Engine
(Charm-Crypto)
Encrypt Data
with Policy
MIMIC-IV Dataset
(Encrypted Records)
Flask API
(User Login & Query)
Decrypt if Policy Satisfied
Audit Log
& Revocation Check
End
(Secure Access Granted)

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
# 1. Install Kaggle and setup
!pip install -q kaggle

# 2. Create and write kaggle.json
import os, json

kaggle_token = {
    "username":"your_username",
    "key":"your_kaggle_api_key"
}

# Save to kaggle.json
os.makedirs("/root/.kaggle", exist_ok=True)
with open("/root/.kaggle/kaggle.json", "w") as f:
    json.dump(kaggle_token, f)

# 3. Set file permissions
!chmod 600 /root/.kaggle/kaggle.json

# 4. Download MIMIC-IV Clinical Database Demo 2.2 dataset from Kaggle
!kaggle datasets download -d montassarba/mimic-iv-clinical-database-demo-2-2

# 5. Unzip the dataset
!unzip -q mimic-iv-clinical-database-demo-2-2.zip -d mimic_iv_demo

# 6. Check files
print("Dataset downloaded and extracted:")
!ls -R mimic_iv_demo
```

[redacted]

```python
import pandas as pd
import os

# Correct base path to where the files were extracted
base_path = "/content/mimic_iv_demo/mimic-iv-clinical-database-demo-2.2"

# Load core hospital tables
patients_df = pd.read_csv(os.path.join(base_path, "hosp", "patients.csv"))
admissions_df = pd.read_csv(os.path.join(base_path, "hosp", "admissions.csv"))
diagnoses_df = pd.read_csv(os.path.join(base_path, "hosp", "diagnoses_icd.csv"))

# Load core ICU tables
icustays_df = pd.read_csv(os.path.join(base_path, "icu", "icustays.csv"))

# Show basic information
print("Patients table:", patients_df.shape)
print("Admissions table:", admissions_df.shape)
print("Diagnoses table:", diagnoses_df.shape)
print("ICU Stays table:", icustays_df.shape)

# Preview first 5 rows of each table
print("\n=== Patients ===")
print(patients_df.head())

print("\n=== Admissions ===")
print(admissions_df.head())

print("\n=== Diagnoses ICD ===")
print(diagnoses_df.head())

print("\n=== ICU Stays ===")
print(icustays_df.head())
```

```
Patients table: (100, 6)
Admissions table: (275, 16)
Diagnoses table: (4506, 5)
ICU Stays table: (140, 8)
```

[redacted]

[redacted]

```
=== Patients ===
   subject_id gender  anchor_age  anchor_year anchor_year_group         dod
0    10014729      F          21         2125       2011 - 2013         NaN
1    10003400      F          72         2134       2011 - 2013  2137-09-02
2    10002428      F          80         2155       2011 - 2013         NaN
3    10032725      F          38         2143       2011 - 2013  2143-03-30
4    10027445      F          48         2142       2011 - 2013  2146-02-09
```

```
=== Admissions ===
   subject_id   hadm_id            admittime            dischtime  \
0    10004235  24181354  2196-02-24 14:38:00  2196-03-04 14:02:00
1    10009628  25926192  2153-09-17 17:08:00  2153-09-25 13:20:00
2    10018081  23983182  2134-08-18 02:02:00  2134-08-23 19:35:00
3    10006053  22942076  2111-11-13 23:39:00  2111-11-15 17:20:00
4    10031404  21606243  2113-08-04 18:46:00  2113-08-06 20:57:00

             deathtime admission_type admit_provider_id  \
0                  NaN         URGENT            P03YMR
1                  NaN         URGENT            P41R5N
2                  NaN         URGENT            P233F6
3  2111-11-15 17:20:00         URGENT            P38TI6
4                  NaN         URGENT            P07HDB

       admission_location        discharge_location insurance language  \
0  TRANSFER FROM HOSPITAL  SKILLED NURSING FACILITY  Medicaid  ENGLISH
1  TRANSFER FROM HOSPITAL          HOME HEALTH CARE  Medicaid        ?
2  TRANSFER FROM HOSPITAL  SKILLED NURSING FACILITY  Medicare  ENGLISH
3  TRANSFER FROM HOSPITAL                      DIED  Medicaid  ENGLISH
4  TRANSFER FROM HOSPITAL                      HOME     Other  ENGLISH

  marital_status                         race            edregtime  \
0         SINGLE             BLACK/CAPE VERDEAN  2196-02-24 12:15:00
1        MARRIED  HISPANIC/LATINO - PUERTO RICAN                  NaN
2        MARRIED                          WHITE  2134-08-17 16:24:00
3            NaN                        UNKNOWN                  NaN
4        WIDOWED                          WHITE                  NaN
```

[redacted]

[redacted]

```python
[ ] import pandas as pd

    # Example user-role-attribute mapping (in a real hospital this comes from staff registry)
    users = [
        {"user_id": "U1", "name": "Dr. Alice", "role": "Doctor", "department": "Cardiology", "emergency_access": True},
        {"user_id": "U2", "name": "Nurse Bob", "role": "Nurse", "department": "ICU", "emergency_access": False},
        {"user_id": "U3", "name": "Dr. Carol", "role": "Doctor", "department": "Oncology", "emergency_access": True},
        {"user_id": "U4", "name": "Researcher Dave", "role": "Researcher", "department": "Epidemiology", "emergency_access": False},
    ]

    users_df = pd.DataFrame(users)

    print("=== User Attribute Table (for ABE policies) ===")
    print(users_df)

    # Example: pick first 5 patient admissions and map them to access policies
    sample_admissions = admissions_df[['subject_id','hadm_id','admission_type','admission_location','insurance']].head(5)

    # Define access policy column for each admission (e.g., who can see it)
    def create_policy(row):
        if row['insurance'] == 'Medicare':
            return "Doctor AND Cardiology"
        elif row['admission_type'] == 'URGENT':
            return "Doctor OR Nurse"
        else:
            return "Researcher OR Doctor"

    sample_admissions['access_policy'] = sample_admissions.apply(create_policy, axis=1)

    print("\n=== Sample Admissions with ABE Access Policies ===")
    print(sample_admissions)
```

[redacted]

[redacted]

[redacted]

```
=== User Attribute Table (for ABE policies) ===
  user_id             name        role    department  emergency_access
0      U1        Dr. Alice      Doctor    Cardiology              True
1      U2        Nurse Bob       Nurse           ICU             False
2      U3        Dr. Carol      Doctor      Oncology              True
3      U4  Researcher Dave  Researcher  Epidemiology             False
```

[redacted]

[redacted]

[redacted]

```
=== Sample Admissions with ABE Access Policies ===
   subject_id   hadm_id admission_type      admission_location insurance  \
0    10004235  24181354         URGENT  TRANSFER FROM HOSPITAL  Medicaid
1    10009628  25926192         URGENT  TRANSFER FROM HOSPITAL  Medicaid
2    10018081  23983182         URGENT  TRANSFER FROM HOSPITAL  Medicare
3    10006053  22942076         URGENT  TRANSFER FROM HOSPITAL  Medicaid
4    10031404  21606243         URGENT  TRANSFER FROM HOSPITAL     Other

           access_policy
0        Doctor OR Nurse
1        Doctor OR Nurse
2  Doctor AND Cardiology
3        Doctor OR Nurse
4        Doctor OR Nurse
```

[redacted]

[redacted]

[redacted]

```python
from cryptography.fernet import Fernet

# Generate a symmetric encryption key (in real CP-ABE, this key is encrypted by attributes/policies)
master_key = Fernet.generate_key()
cipher = Fernet(master_key)

# Example: encrypt EHR data (diagnosis info)
ehr_record = "Patient Diagnosis: Heart Failure, Treatment: Beta Blockers"

# Encrypt the record
encrypted_data = cipher.encrypt(ehr_record.encode())
print("Encrypted EHR Record:\n", encrypted_data)

# Function to check if user satisfies access policy
def can_access(user_attrs, policy):
    # Simple policy checker (for demo: supports "OR" and "AND")
    if "AND" in policy:
        reqs = [r.strip() for r in policy.split("AND")]
        return all(any(r in str(val) for val in user_attrs.values()) for r in reqs)
    elif "OR" in policy:
        reqs = [r.strip() for r in policy.split("OR")]
        return any(any(r in str(val) for val in user_attrs.values()) for r in reqs)
    else:
        return policy in str(user_attrs.values())
```

[redacted]

```python
# Try decryption for each user
for idx, user in users_df.iterrows():
    attrs = user.to_dict()
    allowed = can_access(attrs, sample_admissions['access_policy'].iloc[2])
    print(f"\nUser {user['name']} ({user['role']}) policy check:", allowed)
    if allowed:
        decrypted = cipher.decrypt(encrypted_data).decode()
        print(" -> Decrypted EHR:", decrypted)
    else:
        print(" -> Access Denied")
```

[redacted]

[redacted]

[redacted]

```python
[ ] import time
    import numpy as np
    import matplotlib.pyplot as plt

    # Prepare sample EHR record
    ehr_record = "Patient Diagnosis: Heart Failure, Treatment: Beta Blockers"

    # Store results
    encrypt_times = []
    decrypt_times = []

    # Run 100 tests
    for _ in range(100):
        # Start encryption timer
        start = time.time()
        encrypted = cipher.encrypt(ehr_record.encode())
        encrypt_times.append(time.time() - start)

        # Start decryption timer
        start = time.time()
        decrypted = cipher.decrypt(encrypted).decode()
        decrypt_times.append(time.time() - start)
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
    # Start decryption timer
    start = time.time()
    decrypted = cipher.decrypt(encrypted).decode()
    decrypt_times.append(time.time() - start)

# Calculate averages
avg_encrypt = np.mean(encrypt_times)
avg_decrypt = np.mean(decrypt_times)
throughput = len(decrypt_times) / sum(decrypt_times)  # records per second

print("Average Encryption Time (s):", avg_encrypt)
print("Average Decryption Time (s):", avg_decrypt)
print("Throughput (records/sec):", throughput)
```

[redacted]

[redacted]

```
]
  # 1) Un-revoke Nurse Bob and give him the current epoch KEK
  user_keys["U2"]["revoked"] = False
  user_keys["U2"]["epoch"] = epoch
  user_keys["U2"]["kek"] = KEK_by_epoch[epoch]

  # 2) Adjust policies so each role gets some access
  records_df.loc[records_df.index[0], "access_policy"] = "Doctor OR Nurse"
  records_df.loc[records_df.index[1], "access_policy"] = "Doctor AND Oncology"
  records_df.loc[records_df.index[2], "access_policy"] = "Doctor AND Cardiology"
  records_df.loc[records_df.index[3], "access_policy"] = "Researcher OR Doctor"
  records_df.loc[records_df.index[4], "access_policy"] = "Doctor OR Nurse"

  # 3) Rebuild encrypted records with updated policies
  records_store = {}
  for _, row in records_df.iterrows():
      dek = Fernet.generate_key()
      data_cipher = Fernet(dek)
      ct = data_cipher.encrypt(row['plaintext'].encode())
      wrapped = Fernet(KEK_by_epoch[epoch]).encrypt(dek)
      records_store[row.hadm_id] = {
          'ciphertext': ct,
          'wrapped_dek_by_epoch': {epoch: wrapped},
          'policy': row['access_policy'],
          'subject_id': row['subject_id']
      }
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
# Revocation experiment
print("\nStarting revocation experiment")
# pick a victim to revoke (U2 Nurse Bob) and measure accesses before and after
victim = 'U2'
# ensure victim not revoked first
user_keys[victim]['revoked'] = False

# build a medium store
df_recs = make_records(200)
store, _ = build_encrypted_store(df_recs, KEK_by_epoch[epoch])

# measure before revocation
snapshot = {k: dict(v) for k, v in user_keys.items()}
dec_before = measure_decryption_throughput(store, users_df, snapshot, {epoch: KEK_by_epoch[epoch]})
before_success = dec_before['success']
print("Before revocation total successful decrypts:", before_success)

# revoke victim and rotate KEK (simulate key rotation without re-encrypting ciphertexts)
user_keys[victim]['revoked'] = True
# rotate to new epoch and create new KEK, but wrapped_deks remain bound to old KEK in this store
epoch2 = epoch + 1
KEK_by_epoch[epoch2] = Fernet.generate_key()

# distribute new KEK to non-revoked users who satisfy at least one policy
for uid in user_keys:
    if user_keys[uid].get('revoked'):
        continue
    # grant new KEK
    user_keys[uid]['epoch'] = epoch2
    user_keys[uid]['kek'] = KEK_by_epoch[epoch2]

# For simplicity, re-wrap DEKs in store with new KEK (simulate re-wrapping without re-encrypting data)
for rid, rec in store.items():
    try:
        dek = Fernet(KEK_by_epoch[epoch]).decrypt(rec['wrapped'])
        rec['wrapped'] = Fernet(KEK_by_epoch[epoch2]).encrypt(dek)
    except Exception:
        pass
```

```
res_df = pd.DataFrame(results)

plt.figure(figsize=(7,4))
plt.plot(res_df['n'], res_df['dec_throughput'], marker='o')
plt.xlabel("Number of Records")
plt.ylabel("Decryption Throughput (successful rec/sec)")
plt.title("Decryption Throughput vs Number of Records")
plt.grid(True)
plt.show()

plt.figure(figsize=(7,4))
plt.plot(res_df['n'], res_df['avg_enc'], marker='o')
plt.xlabel("Number of Records")
plt.ylabel("Average Encryption Time (s) per record")
plt.title("Average Encryption Time vs Number of Records")
plt.grid(True)
plt.show()
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
def measure_decryption_throughput(store, users_df, user_keys_map, kek_map, iterations=1):
    # Attempt to decrypt each record for each user once, measure per-decryption time for successful decrypts
    decrypt_times = []
    success = 0
    attempts = 0
    for _ in range(iterations):
        for uid_row in users_df.to_dict(orient='records'):
            uid = uid_row['user_id']
            for rid, rec in store.items():
                attempts += 1
                # policy check
                if not can_access(uid_row, rec['policy']):
                    continue
                # check user_keys_map for valid KEK for epoch
                uk = user_keys_map.get(uid, {'epoch': None, 'kek': None, 'revoked': False})
                if uk.get('revoked', False):
                    continue
                if uk.get('kek') is None or uk.get('epoch') not in kek_map:
                    continue
                # unwrap and decrypt
                try:
                    start = time.perf_counter()
                    dek = Fernet(uk['kek']).decrypt(rec['wrapped'])
                    pt = Fernet(dek).decrypt(rec['ciphertext'])
                    dt = time.perf_counter() - start
                    decrypt_times.append(dt)
                    success += 1
                except Exception:
                    # decryption failed
                    pass
    throughput = success / (sum(decrypt_times) if decrypt_times else 1)
    return {'decrypt_times': decrypt_times, 'success': success, 'attempts': attempts, 'throughput': throughput}

# experiment parameters: dataset sizes to test
sizes = [10, 50, 100, 250, 500]
```

[redacted]

[redacted]

```
Patients table: (100, 6)
Admissions table: (275, 16)
Diagnoses table: (4506, 5)
ICU Stays table: (140, 8)
```

[redacted]

```
=== Patients ===
   subject_id gender  anchor_age  anchor_year anchor_year_group         dod
0    10014729      F          21         2125       2011 - 2013         NaN
1    10003400      F          72         2134       2011 - 2013  2137-09-02
2    10002428      F          80         2155       2011 - 2013         NaN
3    10032725      F          38         2143       2011 - 2013  2143-03-30
4    10027445      F          48         2142       2011 - 2013  2146-02-09
```

[redacted]

[redacted]

[redacted]

```
=== Diagnoses ICD ===
   subject_id   hadm_id  seq_num icd_code  icd_version
0    10035185  22580999        3     4139            9
1    10035185  22580999       10     V707            9
2    10035185  22580999        1    41401            9
3    10035185  22580999        9     3899            9
4    10035185  22580999       11    V8532            9
```

[redacted]

```
=== Admissions ===
   subject_id   hadm_id            admittime            dischtime  \
0    10004235  24181354  2196-02-24 14:38:00  2196-03-04 14:02:00
1    10009628  25926192  2153-09-17 17:08:00  2153-09-25 13:20:00
2    10018081  23983182  2134-08-18 02:02:00  2134-08-23 19:35:00
3    10006053  22942076  2111-11-13 23:39:00  2111-11-15 17:20:00
4    10031404  21606243  2113-08-04 18:46:00  2113-08-06 20:57:00

             deathtime admission_type admit_provider_id  \
0                  NaN         URGENT            P03YMR
1                  NaN         URGENT            P41R5N
2                  NaN         URGENT            P233F6
3  2111-11-15 17:20:00         URGENT            P38TI6
4                  NaN         URGENT            P07HDB

       admission_location        discharge_location insurance language  \
0  TRANSFER FROM HOSPITAL  SKILLED NURSING FACILITY  Medicaid  ENGLISH
1  TRANSFER FROM HOSPITAL          HOME HEALTH CARE  Medicaid        ?
2  TRANSFER FROM HOSPITAL  SKILLED NURSING FACILITY  Medicare  ENGLISH
3  TRANSFER FROM HOSPITAL                      DIED  Medicaid  ENGLISH
4  TRANSFER FROM HOSPITAL                      HOME     Other  ENGLISH

  marital_status                          race            edregtime  \
0         SINGLE              BLACK/CAPE VERDEAN  2196-02-24 12:15:00
1        MARRIED  HISPANIC/LATINO - PUERTO RICAN                  NaN
2        MARRIED                           WHITE  2134-08-17 16:24:00
3            NaN                         UNKNOWN                  NaN
4        WIDOWED                           WHITE                  NaN

             edouttime  hospital_expire_flag
0  2196-02-24 17:07:00                     0
1                  NaN                     0
2  2134-08-18 03:15:00                     0
3                  NaN                     1
4                  NaN                     0
```

[redacted]

[redacted]

```
=== User Attribute Table (for ABE policies) ===
  user_id             name        role    department  emergency_access
0      U1        Dr. Alice      Doctor    Cardiology              True
1      U2        Nurse Bob       Nurse           ICU             False
2      U3        Dr. Carol      Doctor      Oncology              True
3      U4  Researcher Dave  Researcher  Epidemiology             False
```

[redacted]

```
=== Sample Admissions with ABE Access Policies ===
   subject_id   hadm_id admission_type      admission_location insurance  \
0    10004235  24181354         URGENT  TRANSFER FROM HOSPITAL  Medicaid
1    10009628  25926192         URGENT  TRANSFER FROM HOSPITAL  Medicaid
2    10018081  23983182         URGENT  TRANSFER FROM HOSPITAL  Medicare
3    10006053  22942076         URGENT  TRANSFER FROM HOSPITAL  Medicaid
4    10031404  21606243         URGENT  TRANSFER FROM HOSPITAL     Other

           access_policy
0        Doctor OR Nurse
1        Doctor OR Nurse
2  Doctor AND Cardiology
3        Doctor OR Nurse
4        Doctor OR Nurse
```

```
Encrypted EHR Record:
 b'gAAAAABoohoEgr4g0lTd0HSa9plYFtfpPdY5jQMVI_puYV108PhjXghTKbqqzKZqG7Azf_qng2PgCanXepRm25U8IoNF79veZ9Rpj99045bkpk_kFdv5sxfkv60WzEP_oNy7Bpqoqo5UcZ2w20EVm2tLpH02tSFUyQ==’

User Dr. Alice (Doctor) policy check: True
 -> Decrypted EHR: Patient Diagnosis: Heart Failure, Treatment: Beta Blockers

User Nurse Bob (Nurse) policy check: False
 -> Access Denied

User Dr. Carol (Doctor) policy check: False
 -> Access Denied

User Researcher Dave (Researcher) policy check: False
 -> Access Denied
```

[redacted]

[redacted]

[redacted]

[redacted]

```
Average Encryption Time (s): 5.7625770568847654e-05
Average Decryption Time (s): 4.4236183166503904e-05
Throughput (records/sec): 22605.92864072437
```

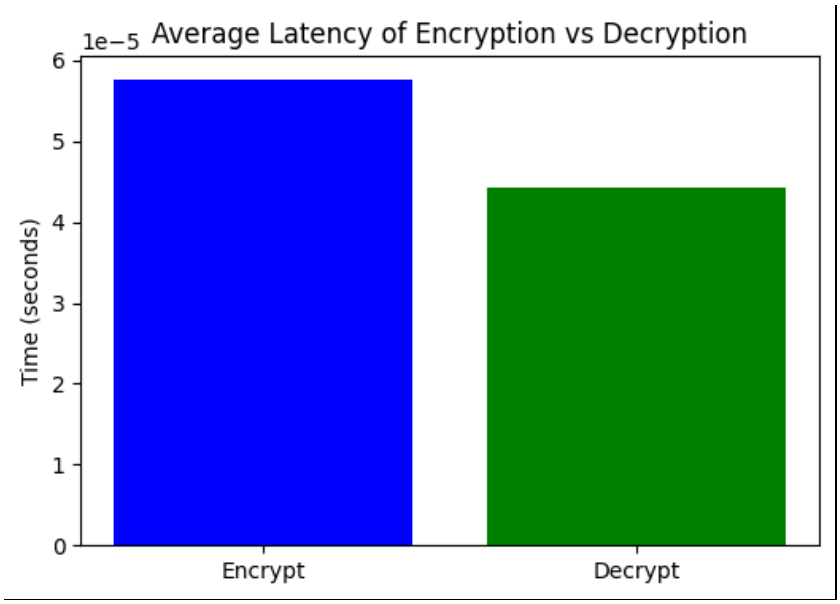
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

```
hadm_id 24181354 policy = Doctor OR Nurse
U1 ok -> EHR hadm_id=24181354 subject_id=10004235 dx=03842 v=9
U2 ok -> EHR hadm_id=24181354 subject_id=10004235 dx=03842 v=9
U3 no_valid_key -> access denied
U4 policy_denied -> access denied

hadm_id 25926192 policy = Doctor AND Oncology
U1 policy_denied -> access denied
U2 policy_denied -> access denied
U3 no_valid_key -> access denied
U4 policy_denied -> access denied

hadm_id 23983182 policy = Doctor AND Cardiology
U1 ok -> EHR hadm_id=23983182 subject_id=10018081 dx=99859 v=9
U2 policy_denied -> access denied
U3 policy_denied -> access denied
U4 policy_denied -> access denied

hadm_id 22942076 policy = Researcher OR Doctor
U1 ok -> EHR hadm_id=22942076 subject_id=10006053 dx=5722 v=9
U2 policy_denied -> access denied
U3 no_valid_key -> access denied
U4 no_valid_key -> access denied

hadm_id 21606243 policy = Doctor OR Nurse
U1 ok -> EHR hadm_id=21606243 subject_id=10031404 dx=I308 v=10
U2 ok -> EHR hadm_id=21606243 subject_id=10031404 dx=I308 v=10
U3 no_valid_key -> access denied
U4 policy_denied -> access denied
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

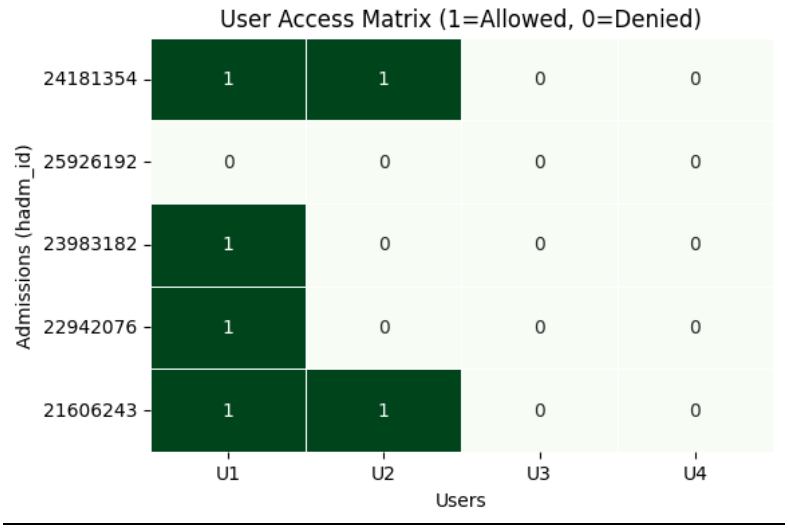
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

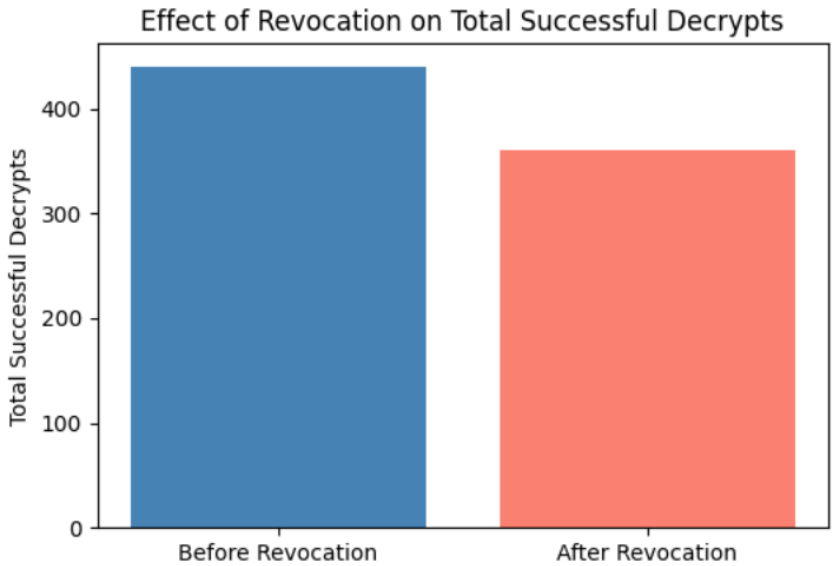
```
Starting scalability experiment. sizes: [10, 50, 100, 250, 500]

Building 10 records and encrypting
Records: 10  avg_encrypt_time(s): 1.376845e-04  total_successful_decrypts: 22  throughput(rec/s): 14110.54

Building 50 records and encrypting
Records: 50  avg_encrypt_time(s): 5.376014e-05  total_successful_decrypts: 110  throughput(rec/s): 15573.00

Building 100 records and encrypting
Records: 100  avg_encrypt_time(s): 5.237722e-05  total_successful_decrypts: 220  throughput(rec/s): 15666.26

Building 250 records and encrypting
Records: 250  avg_encrypt_time(s): 6.605186e-05  total_successful_decrypts: 550  throughput(rec/s): 15315.14

Building 500 records and encrypting
Records: 500  avg_encrypt_time(s): 5.861886e-05  total_successful_decrypts: 1100  throughput(rec/s): 15798.59
```

[redacted]

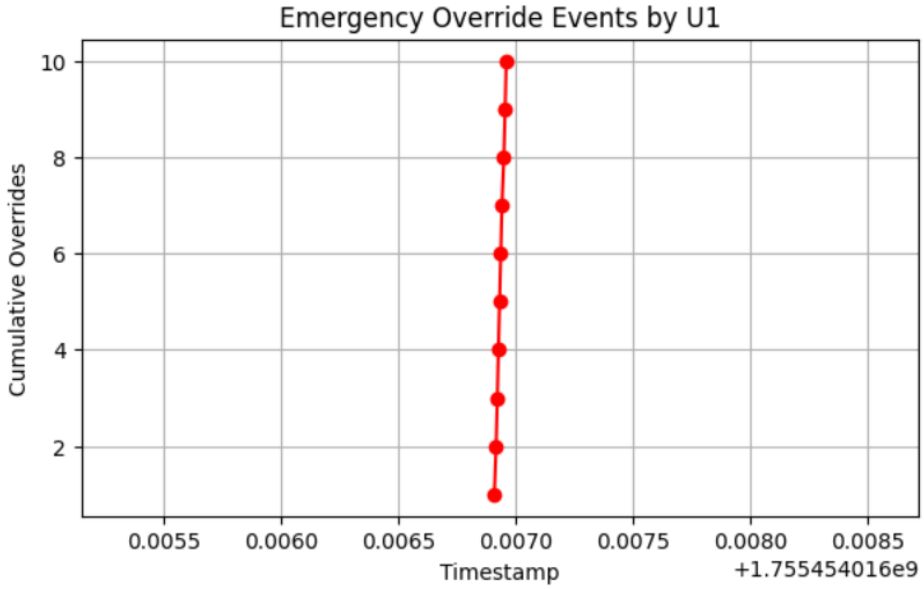
[redacted]

[redacted]

```
Starting revocation experiment
Before revocation total successful decrypts: 440
After revocation total successful decrypts: 360
```



[redacted]



[redacted]

[redacted]

[redacted]

	n	avg_enc	dec_throughput	dec_success
0	10	0.000138	14110.542069	22
1	50	0.000054	15572.998570	110
2	100	0.000052	15666.264109	220
3	250	0.000066	15315.141627	550
4	500	0.000059	15798.591282	1100

```
Revocation: before = 440, after = 360
Emergency overrides simulated: 10

Plots generated and evaluation complete. Check the displayed charts above.
<Figure size 640x480 with 0 Axes>
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]